The Visual Computer (2021) 37:2965–2974
https://doi.org/10.1007/s00371-021-02205-3

ORIGINAL ARTICLE

Grid-induced bounding volume hierarchy for ray tracing dynamic scenes

Satoshi Nishimura[1]

Accepted: 7 June 2021 / Published online: 1 July 2021
© The Author(s) 2021, corrected publication 2025

Abstract
This paper proposes a novel method for accelerating ray tracing of animated scenes in which objects are moved, added, or deleted. The method uses two trees with different structures. The first tree is a hierarchical grid tree that is easily generatable from the primitive list and modifiable according to its change. The second tree is a bounding volume hierarchy (BVH) tree, constructed from the first tree with intra-node agglomerative clustering. When the scene is modified, the first tree is updated, and then, the second tree is partially rebuilt for accessed nodes in the first tree. The resulting BVH tree is deterioration-free; it always coincides with the tree generated by frame-wise rebuilding. For applications where a small portion of the scene is modified between frames, the tree formation cost of the proposed method is much lower than frame-wise rebuilding. Experiments showed that the quality of BVH trees generated by this method is comparable to those created by the binned surface area heuristic approach, and that the tree formation speed is faster not only for dynamic scenes but also for static scenes.

Keywords Ray tracing · Bounding volume hierarchy · Hierarchical grids · Agglomerative clustering

1 Introduction

Beside its excellent capability for handling complex lighting phenomena, ray tracing has a noticeable property in its rendering performance. With an appropriate precomputed data structure for acceleration, the rendering time in ray tracing is sublinear against the number of objects in the scene [1, 2]. This implies that ray tracing offers efficient image generation for large-scale scenes without relying on complicated culling techniques and can therefore be a good alternative for polygon rendering in interactive applications like computer games.

Ray tracing, however, has so far been considered to have difficulties in efficient management of the acceleration data structures for dynamic scenes, in which objects are moved, deformed, newly added, and deleted between frames. The following aspects should be considered carefully in such applications.

Good tracing performance. Looking at recent projects on ray tracing software packages [3, 4], the current baseline of acceleration structure for static scenes seems to be the bounding volume hierarchy (BVH) constructed with surface area heuristic (SAH) cost optimization [5, 6]. With such acceleration, interactive-rate rendering can be achieved. We would also like to keep similar tracing performance for dynamic scenes.

Fast construction/modification of the tree structure. The simplest way to handle dynamic scenes is to rebuild acceleration structure from scratch for every frame (e.g., [6, 7]); however, this requires linear computational cost at the least and, therefore, the aforementioned advantage of ray tracing regarding its performance would be lost. In the situation where a part of scene geometry is modified, the cost for updating the tree structure should ideally not be affected too much by the total number of objects in the structure.

No restrictions on scene modification. Some methods impose restrictions on the type of scene modification. For example, the refitting approach [5, 8, 9] is applicable for object moves but not for insertion or deletion of objects.

✉ Satoshi Nishimura
nisim@u-aizu.ac.jp

[1] School of Computer Science and Engineering, The University of Aizu, Aizuwakamatsu, Fukushima 965-8580, Japan

Stability of performance. In interactive applications, frame rate should be kept constant as much as possible. Incremental tree updates (e.g., [5, 8–10]) often cause unpredictable deterioration of the tree quality, thus affecting the flatness of the frame rate. Although there are techniques that measure tree quality and fix the tree occasionally [8, 9, 11], they might lead to an unexpected pause in an interactive session.

This paper proposes a new method for constructing and updating BVH trees for dynamic scenes with arbitrary types of scene modification. While it uses a hierarchical grid tree as a base structure, it eventually generates an ordinary BVH tree, and thus, standard traversal engines can be used in rendering. The method constructs a BVH tree from scratch in $\mathcal{O}(N \log N)$ time, and updates the tree in $\mathcal{O}(M \log N)$ time, where N and M represent the total number of primitives and the number of scene modifications (i.e., primitive insertions and deletions), respectively, and therefore, the tree update cost for dynamic scenes with few updates is much lower than that of rebuilding approaches. The quality of the resulting BVH tree is competitive compared to that created by the binned SAH approach [6]. There is no temporary degradation of the tree quality; the generated BVH tree is always "fresh" in the sense that it coincides with the one generated by frame-wise rebuilding.

2 Background

Various acceleration data structures for ray tracing have been proposed so far. Wald et al. [12] give a good survey of related techniques, except for recent ones. Commonly known acceleration structures are grids, BVH trees, and kd-trees.

2.1 Grid-based approaches

Grid-based approaches [13, 14] decompose the scene space into cells at regular intervals. For adapting to object coherence in the scene, grids are often organized hierarchically [15]. This hierarchical grid structure can be viewed as the generalization of the octree.

An advantage of grids in the context of dynamic scenes is that it is easy to incrementally insert or delete objects, because, once the scene space for an animation is defined, relative positions among objects and grid cells are fixed. Reinhard et al. [16] discusses a constant-time algorithm for inserting an object to a hierarchical grid by storing objects not only to leaf cells but also to cells at an intermediate level.

A disadvantage of grids lies in their ray tracing performance; it is generally recognized that grids are slower than other approaches. The author also tried the grid structure directly for acceleration, but could not obtain performance better than moderately optimized BVH trees. Two reasons can be considered for this. One is that much computational effort is demanded for traversing empty cells. The other is due to the duplication of intersection calculations against objects, attributed to multiple references from grid cells to objects. Such redundancy can be avoided using *mailboxes* [14]; however, they have data locking problems in parallel implementation.

2.2 Bounding volume hierarchy

Bounding volume hierarchy accelerates ray tracing through a nested structure of bounding volumes, most commonly, axis-aligned bounding boxes (AABBs). In the early period of ray tracing, BVH trees have been manually constructed based on scene graphs, but later more efficient trees came to be generated automatically. For static scenes, one of the well-known construction methods is the sweep algorithm [5], which recursively subdivides the set of objects with a split plane that yields the best estimated SAH cost among candidates positioned at the centroid of every object.

To handle dynamic scenes, the easiest way is to construct the tree from scratch for every frame. This requires a faster construction algorithm. The binned SAH algorithm [6], which restricts the splitting candidates to several positions at regular intervals, is much faster than the sweep algorithm while the degradation of the tree quality is known to be small. Lauterbach et al. [7] proposed an even faster algorithm called LBVH, which utilizes Morton code for constructing trees based on spatial median splits. Pantaleoni and Luebke [17] proposed a method called HLBVH, which uses LBVH for the bottom part of the tree and applies the sweep algorithm for the top part. Garanzha et al. [18] improved the efficiency for finding good split planes in GPU implementation using an auxiliary grid structure. This grid structure is transient data prepared for each frame and not like the one used in this paper, which is kept over frames.

Since the quality of trees generated by LBVH or HLBVH is somewhat lower than the sweep algorithm, there is motivation for the development of strategies for improving the tree quality. Ganestam et al. [19] succeeded in generating trees a little better than the sweep algorithm with less construction time by forming mini-trees for object groups and constructing the top part of the tree using them. For optimization, mini-trees with bad quality are broken into finer ones. Karras and Aila [20] proposed a method for improving the quality of an existing tree by forming a *treelet*, a small subset of nodes itself forming a tree, from each node as its root and applying dynamic programming for optimizing the structure. Domingues and Pedrini [21] achieved faster construction by replacing dynamic programming with fast agglomerative clustering [22]. Meister and Bittner [23] applied a faster agglomerative clustering algorithm to the entire tree, by which a

high-quality tree is generated at a reasonable construction speed with GPUs.

When only a partial set of objects are moved or deformed in dynamic scenes, the refitting technique [5] is a possible solution. This technique updates the AABBs of the BVH tree nodes related to the scene change, but keeps the same tree structure. This enables quick updating but often leads to significant degradation of tree quality if the degree of scene change is large. Therefore, various methods for recovering the quality have been proposed. Lauterbach et al. [11] proposed to monitor tree quality based on the surface areas of AABBs and rebuild the tree when the quality goes under a threshold. Larsson and Akenine-Möller [8] proposed to compute a volume-based efficiency measure for each updated node and lazily reconstruct those subtrees whose measure values are below a threshold. Yoon et al. [9] selectively restructure the tree by checking the overlaps of AABBs as well as restructuring benefit for node pairs. Kopta et al. [10] discuss tree rotations as well as the split and merge of leaf nodes after refitting. Note that these methods allow temporary degradation of the tree quality, and therefore, the stability of trace performance is not well guaranteed. Moreover, the refitting technique cannot be used for scenes where new objects are added or objects are removed.

The incremental BVH tree construction proposed by Bittner et al. [24] might be applicable for dynamic scenes. They successively add an object to the tree by using a branch-and-bound method for finding the best position in terms of global SAH. Occasionally, the tree is refined for better quality. Unfortunately, the branch-and-bound search is computationally expensive, and moreover, the tree quality is possibly instable, since the resultant tree varies depending on the order of object insertion.

2.3 Kd-trees

Kd-trees recursively subdivide the scene space with axis-aligned split planes. Fast construction methods for kd-trees are known [25]. Kd-trees had once been believed to be the best acceleration structure; however, they seem to have been replaced by BVH trees recently. One reason for this is that the refitting technique for dynamic scenes can only be used in BVH trees. Another possible reason is that the duplicated intersection calculation issues, as in grids, also exist in kd-trees. As for refitting, Yang et al. [26] recently proposed an alternative approach that redistributes primitives while keeping the same tree structure but optionally expanding leaf nodes if their size becomes large as the result of the redistribution. This approach is faster than rebuilding the tree; however, it leads to the degradation of the tree quality.

3 Grid-induced boundary volume hierarchy

As addressed in the previous section, the acceleration structure that provides the easiest way of incremental modification for dynamic scenes is a grid; however, using the grid directly for rendering results in poor performance. To achieve tracing performance comparable to high-quality BVH trees, this paper proposes a method that uses a hierarchical grid as a base structure and translates it into a BVH tree. When a partial set of objects in the scene is modified, the method first updates the grid and then reconstructs the BVH tree only for the modified part, by which efficient updates are achieved. The method is named Grid-induced Boundary Volume Hierarchy (GBVH).

3.1 Outline of the method

GBVH uses a pair of trees: a grid tree and a BVH tree (Fig. 1). These two trees share the same set of leaf nodes, each of which contains a small number of primitives (typically triangles). During generation of an animation, both of the trees stay in memory. For accelerating intersection calculation in the tracing phase, only the BVH tree is consulted.

The grid tree represents a hierarchical grid that recursively subdivides the scene space (Fig. 1a, b). Each interior node in the grid tree is called a grid node. This method assumes that a bounding box (not necessarily being a tight bounding box) of the whole animated scene is known in advance. The top-level grid subdivides such a bounding box into cells (typically at 4^3 resolution). Then, similarly to the hierarchical grid proposed by Jevans and Wyvill [15], each cell that contains a large number of primitives is subdivided further. While their grid structure allows multiple references to each primitive, the grid tree here is based on each primitive's centroid and therefore only has a unique reference to each primitive. This makes the construction or update of the tree much easier.

The BVH tree (Fig. 1c) is constructed based on the grid tree. Each interior node in the BVH tree is called a BVH node. As the figure shows, each grid node corresponds to a disjoint subset of BVH nodes that forms a treelet[1] in the BVH tree. The connection structure between grid nodes is preserved as the connections between treelets in the BVH tree.

When the BVH tree is constructed for frame 0 (i.e., the beginning frame) of an animation, a full grid tree is built, and then, each grid node is translated to its corresponding treelet to form a new BVH tree. Between frame i and frame $i+1$, the grid tree is first updated for every primitive modification between the frames. During this

[1] Treelets here are defined as connected subgraphs consisting of BVH nodes and being slightly different from those defined in [20] which also include leaf nodes.

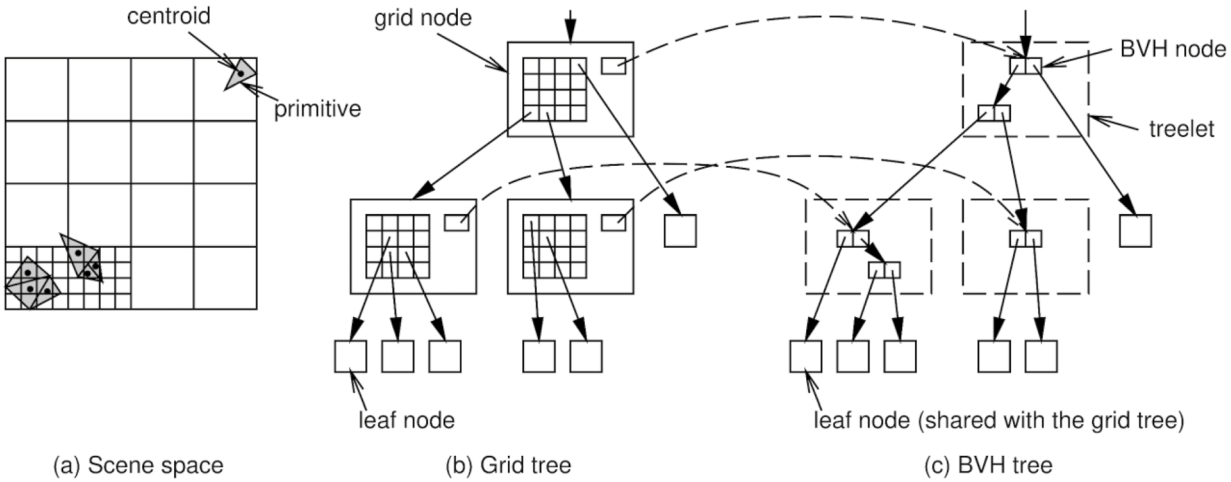


Fig. 1 Structure of the GBVH trees

process, grid nodes that lie in the paths from the root node to leaf nodes containing modified primitives are marked as "dirty". Then, such dirty grid nodes are retranslated to treelets in the BVH tree. For a non-dirty grid node, whose descendants should also be non-dirty, its corresponding subtree in the BVH tree of frame i is reused in frame $i+1$. For this purpose, each grid node keeps a pointer to the root of its corresponding treelet, as depicted by the dotted arrows in Fig. 1.

3.2 Node data structure

The data structure of each type of tree nodes is shown in Table 1. The grid node contains a cell-structured pointer array, a dirty bit, and a pointer to the root of the treelet described in the previous subsection. In addition, the grid node keeps the total number of primitives in the grid, used in the update process of the grid tree, as well as an axis-aligned bounding box enclosing the primitives.

The BVH node consists of pointers to child nodes and bounding boxes, with one bounding box for each child. Storing the bounding box corresponding to a BVH node to its parent node instead of the node itself helps the SIMD implementation of bounding box tests [4]. This paper assumes the use of a binary BVH tree for its simplicity and flexibility, but in principle, the proposed strategy can be applied for other branching factors.

The bounding box stored in a grid or leaf node is valid only if the node's dirty flag is cleared. When a grid or leaf node is newly created, the dirty flag is initialized as set (i.e., dirty).

Table 1 Data contained in the tree nodes

Name	Data type	Description
Grid node		
cells	3D array of node pointers	Pointers to child grid or leaf nodes
nprims	integer	Total number of primitives in this grid
aabb	bounding box	Bounding box enclosing all the primitives in this grid
dirty	Boolean	Dirty flag
bvhnode	pointer	Pointers to the root of the corresponding treelet
BVH node		
left, right	node pointers	Pointers to child nodes
aabbs	two bounding boxes	Bounding boxes for left and right nodes
Leaf node		
primitives	array	Array of primitive data
dirty	Boolean	Dirty flag
aabb	bounding box	Bounding box enclosing all the primitives in this leaf

3.3 Building and updating the grid tree

While a hierarchical grid structure for accelerating ray tracing is known [15], it seems that its incremental update algorithm has not been described in past literature. This subsection presents an incremental update algorithm for the grid tree, in which the tree dynamically grows or shrinks in response to scene modification.

In this paper, only two basic operations for manipulating the grid tree are defined, that is, inserting a primitive to the tree and deleting a primitive from the tree. Building of the initial tree is done by inserting all the primitives in the tree to an empty tree. When a primitive is moved or deformed, it is first deleted and then reinserted.

The insertion and deletion procedures are shown in Fig. 2. These procedures work so that the number of primitives in each leaf node does not exceed a predetermined constant N_{leaf}, unless the level (i.e., distance from the root) of the leaf node reaches a limit $L_{\max}$. (Leaf nodes at level $L_{\max}$ may thus contain more than N_{leaf} primitives.) Limiting the height of the tree is necessary for guaranteeing the termination of the insertion procedure.

The detailed behavior of the above procedures is as follows. When a primitive is inserted to a grid tree, it is first checked whether the tree is empty, and if so, a leaf node is created (lines 5–6). If the root of the tree is a grid node, the primitive counter is incremented (line 8), the dirty flag is set (line 9), and the insertion procedure is recursively called against the cell that contains the centroid of the primitive. If the root of the tree is a leaf node, the primitive is first inserted to the leaf and when the resultant leaf node is too large (i.e., breaking the conditions described above), it is converted to a grid node by recursively calling the procedure (lines 15–18). When a primitive is deleted from the tree, the type of the tree root is first checked (line 22). If it is a grid node, its status is updated, and the deletion procedure is recursively called (lines 23–25). Then, if the number of primitives in the grid goes below the threshold, the grid node is converted to a leaf node or an empty tree (lines 26–31). If the root of the tree is a leaf node upon deletion, the primitive is directly deleted from the node (lines 33–36). Note that the structure of the grid tree obtained by these procedures, ignoring the order of primitives in each leaf node, is uniquely determined by the current set of primitives in the tree; it does not depend on its update history (for example, insertion order) up to that point.

```
1:  void insert_primitive(
2:              Node* &v,     // target node for insertion
3:              p,            // primitive to be inserted
4:              int i):       // level in the grid tree
5:    if v is NULL:
6:      v = a new leaf node just having p
7:    else if v is a grid node:
8:      v->nprims++
9:      v->dirty = true
10:     insert_primitive(v->cells[index of the cell that
                    contains the centroid of p], p, i+1)
11:   else: // v should be a leaf node
12:     add p to v->primitives
13:     v->dirty = true
14:     if |v->primitives| > N_leaf and i < L_max:
15:       w = a new empty grid node
16:       foreach primitive q in v->primitives:
17:         insert_primitive(w, q, i)
18:       v = w

19: void delete_primitive(
20:             Node* &v,     // target node for deletion
21:             p):           // primitive to be deleted
22:   if v is a grid node:
23:     v->nprims--
24:     v->dirty = true
25:     delete_primitive(v->cells[index of the cell that
                    contains the centroid of p], p)
26:     if v->nprims == 0:
27:       v = NULL
28:     else if v->nprims ≤ N_leaf :
29:       w = a new empty leaf node
30:       insert every primitive below v to w
31:       v = w
32:   else: // v should be a leaf node
33:     delete p from v->primitives
34:     v->dirty = true
35:     if v->primitives is empty:
36:       v = NULL
```

Fig. 2 Procedures for manipulating the grid tree. Calling insert_primitive(*root*, *p*, 0) inserts a primitive *p* to the grid tree, while calling delete_primitive(*root*, *p*) deletes *p* from the grid tree, where *root* is the root of the grid tree. Steps for freeing garbage nodes are omitted in the above procedures

3.4 Generating the BVH tree

Once the construction or update of the grid tree is completed, the BVH tree is generated by translating each grid node marked as dirty to its corresponding treelet in the BVH tree. This process should be done in bottom-up order in the grid tree, because forming a treelet for a grid node requires valid pointers to the roots of treelets corresponding to the grid node's children.

Constructing a treelet from each grid node is the same process as standard BVH tree construction from a set of primitives except that what was a primitive in the standard BVH construction is now a child node in the grid tree. Therefore, we have the freedom to choose an appropriate algorithm from existing ones. Since the number of cells in each grid is limited (typically 64), we can afford to use methods more expensive than standard $\mathcal{O}(N \log N)$ algorithms.

Good candidates are the sweep algorithm [5] and agglomerative clustering [21, 22]. In this paper, agglomerative clustering is chosen because of its simplicity and its suitability with regard to computational costs.

The procedures for generating the BVH tree are shown in Fig. 3. The build_BVH procedure generates the BVH tree by calling the build_treelet subprocedure to translate each dirty grid node to its corresponding treelet in the BVH tree. The build_BVH procedure, as side effects, updates the axis-aligned bounding boxes stored in dirty nodes, sets the bvhnode pointers, and clears all the dirty bits in the grid tree.

The build_BVH procedure works with the depth-first traversal of the grid tree. When it encounters a leaf node during the traversal, the bounding box is recalculated if it is dirty, and then the leaf node itself is returned because we want to share it with the BVH tree (lines 2–6). When the procedure encounters a non-dirty grid node, it just returns the BVH subtree stored in the bvhnode pointer (line 17) and thus traversal below the node is omitted. When a dirty grid node is encountered, the procedure first traverses its child nodes to obtain their BVH subtrees (lines 12–14). Then, using this information, the build_treelet subprocedure is called for generating the corresponding treelet (line 15). Note that only the subtrees for non-empty cells are passed to the subprocedure for reducing the problem size there. The generated treelet is stored in the grid node for later use (line 15) and is returned by the procedure (line 17).

```
1:  build_BVH(Node* v):  // v is a node in the grid tree
2:    if v is a leaf node:
3:      if v->dirty:
4:        recalculate v->aabb from v->primitives
5:        v->dirty = false
6:      return (v, v->aabb)
7:    else: // v should be a grid node
8:      if v->dirty:
9:        // L is a set of pairs, each consisting of a BVH
10:       // node (root of a subtree) and an AABB
11:       L = ∅
12:       foreach non-empty cell c in v->cells:
13:         (subtree, aabb) = build_BVH(c)
14:         add (subtree, aabb) to L
15:       (v->bvhnode, v->aabb) = build_treelet(L)
16:       v->dirty = false
17:     return (v->bvhnode, v->aabb)

18: build_treelet(L):
19:   while |L| > 1:
20:     mindist = ∞
21:     foreach element pair (x, y) in L:
22:       if D(x, y) < mindist:
23:         mindist = D(x, y)
24:         (x', y') = (x, y)
25:     w = a new BVH node constructed from (x', y')
26:     remove x' and y' from L
27:     add (w, tight AABB enclosing both x' and y') to L
28:   return the element in L
```

Fig. 3 Procedures for generating the BVH tree. After the manipulation of the grid tree, build_BVH(*root*) is called for each frame to obtain the BVH tree, where *root* is the root of the grid tree. Both the above procedures return a pair of a BVH node and an axis-aligned bounding box (AABB) corresponding to the node

In the build_treelet subprocedure, the BVH treelet is constructed using the agglomerative clustering algorithm proposed by Domingues and Pedrini [21]. Starting with an initial set of clusters (each of which represents a subtree in the BVH tree), in each step of the loop shown in lines 19–27, a cluster pair which has the minimum distance is found, and then such a pair is merged into a single cluster. This process is continued until all the clusters are merged into one. The distance of two clusters x and y, denoted by $D(x, y)$, is defined as the surface area of the tight axis-aligned bounding box enclosing both x and y. The treelet is constructed according to the merging process.

Since the BVH nodes are allocated and freed more frequently than other types of nodes, allocating them individually might be inefficient. The number of BVH nodes in the treelet equals the initial number of clusters minus one. Therefore, BVH nodes can be allocated in a batch once the actual number of nodes is determined (i.e., just before calling build_treelet at line 15).

3.5 Parallelization

This subsection shows how to parallelize the algorithms described in the previous subsections using multiple threads on a multi-core CPU. GPU implementation might also be possible, but I leave this as a topic for future work.

The generation of the BVH trees (Fig. 3) can be parallelized easily. Since the subtree generation process for each cell in a grid is independent, it is possible to parallelize the loop starting at line 12. In the author's implementation, using the tasking mechanism of OpenMP, a task is generated for each instance of the loop body, and task synchronization is done at the end of the loop. (To avoid over-generation of tasks, tasks are created only for the first two levels of grids.)

The parallelization of the grid construction and update (Fig. 2) is less obvious. Such parallelization is achieved by processing multiple insertion/deletion requests at a time. To make the processing easier, grid nodes until some level L_p are created even if the primitive count of such a node is not greater than N_{leaf} (in the evaluation in Sect. 4, $L_p = 1$, that means one level below the root grid). Then, insertion/deletion requests are first classified into cells in those grids according to the centroid of the target primitive in each request. Then, cell-wise parallel invocation of insert_primitive or delete_primitive (depending on each request's type) is performed. OpenMP tasks are used for such invocation in the author's implementation.

3.6 Computational complexity

It is not hard to show that the average-case height of the grid tree containing N primitives is $\mathcal{O}(\log N)$, and therefore, the complexity of inserting or deleting M primitives into the grid tree is $\mathcal{O}(M \log N)$, as long as $L_{\max}$ described in Sect. 3.3 is set so that it is not reached for most primitives.

The computational cost for translating the grid tree to the BVH tree is as follows. Since the number of cells for each grid node is fixed, the translation cost for each node is $\mathcal{O}(1)$. When M insertions/deletions are applied, the number of dirty nodes in the grid tree is $\mathcal{O}(M \log N)$, because the number of leaf nodes to be updated is at most M and the number of grid nodes on the path from the root to each leaf node is $\mathcal{O}(\log N)$. Therefore, the total translation cost is $\mathcal{O}(M \log N)$. The summed-up cost for grid tree manipulation and translation is also $\mathcal{O}(M \log N)$.

4 Evaluation results

This section describes evaluation results for the GBVH method. For this purpose, the author developed a simple ray tracer with the C++ language. The Embree ray tracing kernels (version 3.9.0) [4] were used for ray-triangle intersection, ray-box intersection and basic SIMD vector operations. OpenMP directives were used for multi-threading. Tracing performance was measured with simple diffuse shading (with textures for some scenes) and shadows, and with reflection/refraction rays for some surfaces. The parameters for GBVH were set as follows: each grid node contains 4^3 cells, $N_{\text{leaf}} = 8$, and $L_{\max} = 5$. All results were obtained using an Intel i7-4790 3.6 GHz processor (8 threads) with 16 Gbytes of memory running under the 64-bit Ubuntu 16.04 Linux operating system. The scenes shown in Fig. 4 were used in the evaluation.

4.1 Performance for static scenes

For evaluating the construction speed and quality of trees for generating a single-frame image, the GBVH algorithm was compared with the binned SAH [6] and LBVH [7] algorithms. The author also implemented these algorithms with C++, Embree, and OpenMP in the spirit of keeping the code optimization level the same as GBVH. For the binned SAH algorithm, 16 bins were used for finding split planes, and task-based parallelization was applied for recursively constructing trees. The recursion was terminated if the estimated cost of creating a BVH node is higher than that of creating a leaf node, or the tree depth reaches 30. For computing the SAH cost, $K_{\text{T}} = 1$ and $K_{\text{I}} = 0.2$ were used. The LBVH algorithm was implemented with parallel radix sort provided by Embree for sorting primitives by Morton codes and with task-based parallelization for recursive tree construction. Its recursion was terminated if the number of primitives becomes 8 or less, or the tree depth reaches 30.

Table 2 shows the results for generating an image by constructing a BVH tree from scratch. All the results were measured with the views shown in Fig. 4. Observing the tracing speeds, the quality of the BVH trees generated by the GBVH method is considered to be comparable to that of the binned SAH method and better than that of the LBVH approach. For four out of eight scenes, GBVH yielded the best tracing speeds. However, for the large-sized, powerplant scene, the quality of the tree from the GBVH method is slightly lower than that from the binned SAH approach. Since the form of a GBVH-generated BVH tree is somewhat restricted by the grid tree, GBVH has less freedom in this aspect than the binned SAH approach. It is considered that the degree of such restriction becomes more significant for a larger scene, because the relative size of each treelet against the whole tree becomes smaller. Changing the size of treelets

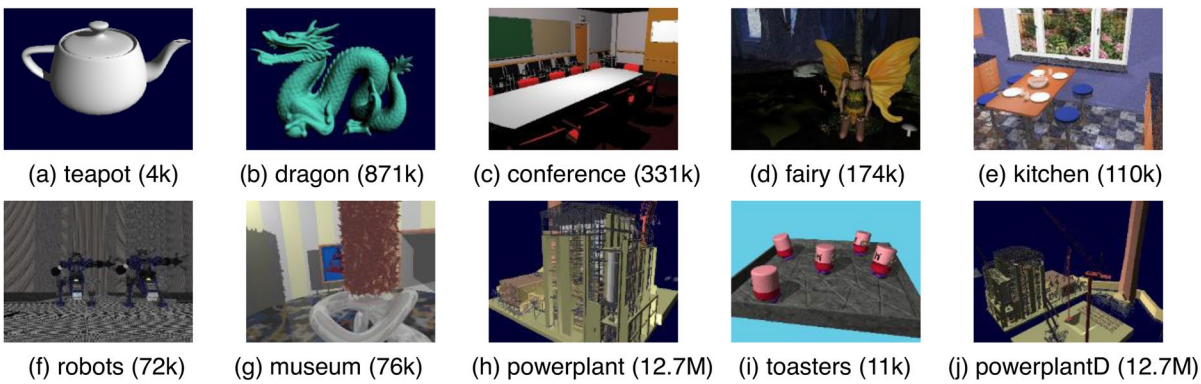


Fig. 4 Test scenes used in the experiments. Scenes (e)–(g) are from the BART [27]. Scene (j) is a dynamic scene using the Power Plant model (http://gamma.cs.unc.edu/POWERPLANT/) where the crane located near the center of the image rotates. The parenthesized numbers show triangle counts in the scenes

Table 2 Performance comparison for static scenes

	teapot	dragon	conference	fairy	kitchen	robots	museum	powerplant
Trace speed (Mrays/s)								
BinSAH	**41.8**	**28.1**	19.9	16.2	20.1	15.2	**11.4**	**9.8**
LBVH	39.2	25.0	15.7	14.4	16.1	13.9	10.2	7.6
GBVH	38.8	23.3	**21.0**	**16.4**	**22.9**	**15.8**	10.1	8.4
Build time (ms)								
BinSAH	0.39	145.1	48.5	25.7	17.7	8.9	12.5	3355.3
LBVH	**0.38**	**49.3**	**16.6**	**10.0**	**6.5**	**4.4**	**5.2**	**695.4**
GBVH	1.01	82.4	23.4	15.4	11.7	5.9	15.5	801.9
	(0.55)	(51.8)	(18.2)	(9.6)	(7.2)	(3.9)	(5.6)	(664.8)
Total time (ms)								
BinSAH	**33.85**	183.5	152.1	156.6	461.0	262.1	**447.2**	3465.8
LBVH	36.10	**92.5**	147.6	157.7	559.4	280.7	494.8	**838.2**
GBVH	37.04	128.9	**121.2**	**144.6**	**401.4**	**248.7**	508.1	930.6

The trace speeds were measured with shading and ray generation. The non-parenthesized build times of GBVH show total times for tree construction, while the parenthesized values show times only for grid tree construction. Each of the total times shown in the bottom rows is the sum of the build time and tracing time for generating a 1200×900 image with one sample/pixel. The bold-faced numbers indicate the best results

adaptively might be effective, but the author leaves this topic to future work.

Interestingly, tree construction in the GBVH method tends to be faster than that in the binned SAH approach. GBVH is more than 4 times faster than the binned approach for the powerplant scene and is approximately 30% faster on average. This implies that, even for the case of rebuilding whole trees for each frame, GBVH still has a performance advantage. In the binned SAH approach, many primitives are directly involved in the decisions of partitioning planes at the top part of the tree. On the other hand, the GBVH method uses only the bounding box information of inferior nodes, and therefore, much less computation is required in the top part of the tree.

4.2 Performance for dynamic scenes

The evaluation of the GBVH method for dynamic scenes was performed by comparing it with two typical methods for handling dynamic scenes: (a) a method where the initial BVH tree is constructed by the binned SAH algorithm, and then the tree is updated with BVH refitting and (b) a method where the BVH tree is rebuilt every frame with the LBVH algorithm. Table 3 shows the results for generating animations using dynamic scenes shown in Fig. 4.

Observing the trace speeds, it was turned out that the GBVH method constantly provides good-quality BVH trees (in the powerplantD scene, however, the binned SAH approach was slightly better, which was probably due to the same reason described in the previous subsection). For the museum scene from the BART benchmark [27], the tree quality is extremely degraded in the case of the refitting approach. This is because the scene contains many primitives moving largely and randomly (related discussions are found in [5]).

As for the times for the construction and update of trees, the results show that the refitting approach is generally faster than the other approaches. However, since the whole tree is traversed for bounding box updates in the refitting approach, the GBVH method, which traverses only the updated part of the tree, becomes superior for the large-sized, powerplantD scene. Comparing the LBVH-based rebuilding approach and the GBVH method, the GBVH method is faster in the case when the percentage of moving primitives is low (i.e., M described in Sect. 3.6 is sufficiently smaller than N). For example, for the kitchen and powerplantD scenes, the GBVH method is more than 3.5 and 63 times faster than the rebuilding method, respectively. If the percentage of moving primitives is high, the tree rebuilding approach with the LBVH-based rebuilding method is more efficient in tree building, but the quality of generated trees is lower as mentioned above.

Comparing the total time for generating animations, the GBVH method outperforms the other two methods for all the tested dynamic scenes. The instability of trace performance as found in the refitting approach is not observed in the GBVH method.

The percentage of dirty nodes shown in Table 3 is the average ratio of the number of dirty grid and leaf nodes to the total number of nodes in the grid tree. This value is considered to indicate how the incremental update in GBVH performs better than complete rebuilding. As the results show, it is highly related to the percentage of the moving primitives in the scene.

Table 3 Performance comparison for dynamic scenes

	fairy	kitchen	robots	museum	toasters	power-plantD
No. of frames	201	420	500	165	246	73
% of moving primitives	66.1%	6.5%	87.1%	85.9%	93.5%	0.009%
% of GBVH dirty nodes	50.8%	6.5%	75.4%	84.6%	95.8%	0.085%
Trace speed (Mrays/s)						
BinSAH-refit	7.9	20.3	14.2	0.3	19.3	**14.8**
LBVH-rebuild	13.4	15.5	12.8	17.7	21.3	11.8
GBVH	**15.2**	**21.3**	**15.8**	**19.4**	**26.3**	11.9
Build time + update time (s)						
BinSAH-refit	**2.27**	**0.68**	**2.04**	**0.81**	**0.16**	10.27
LBVH-rebuild	3.26	4.17	3.28	1.27	0.36	57.30
GBVH	5.60	1.16	4.94	2.87	0.51	**0.88**
	(4.67)	(0.74)	(3.96)	(1.90)	(0.40)	(0.74)
Total time (s)						
BinSAH-refit	55.77	255.27	144.28	2351.55	21.11	15.60
LBVH-rebuild	34.99	337.41	160.35	39.47	19.31	64.00
GBVH	**33.61**	**243.80**	**132.87**	**37.76**	**15.88**	**7.49**
Peak memory usage (Mbytes)						
Grid nodes	2.6	2.5	0.73	2.5	0.29	29
BVH nodes	2.7	2.5	0.77	3.7	0.31	20
Leaf nodes	9.0	6.1	3.5	5.2	0.64	571

The definitions of the trace speed, the build time, and the total time are the same as those of Table 2 except that values for the whole animations are shown

4.3 Memory requirements

Since the GBVH method uses two different indexing structures simultaneously, the increase of memory requirements might be a concern. The bottom rows of Table 3 show the peak memory usage for each category of nodes in the GBVH method. These are the "net amount" sizes of the information shown in Table 1 and do not include implementation-dependent overheads such as unused entries in primitive arrays, unused parts in SIMD vectors, or memory management information.

As can be seen from the table, the total memory requirements for grid nodes are close to those for BVH nodes. Thus, the amount of memory required for indexing structures could be doubled in comparison to methods that uses BVH trees only. However, since a quite large percentage of memory is used by leaf nodes, which have the geometry information of primitives, such an increase is generally insignificant in practical applications. Moreover, since the grid tree is not accessed in the tracing phase, performance degradation due to the increase of memory access bandwidth should not occur.

5 Conclusion

This paper has described a novel method called Grid-induced Boundary Volume Hierarchy for accelerating ray tracing of dynamic scenes allowing arbitrary forms of scene modifications. When a small fraction of the scene was modified, the method showed more than an order of magnitude improvement in the speed of tree formation while keeping time increase in the tracing phase to a negligible level. Such improvement would have a significant impact on the total performance for large-scale scenes and should be even more emphasized if hardware acceleration is employed in the tracing phase.

In future work, the performance of the GBVH method on GPUs should be investigated. In addition, the generation of better BVH trees especially for large-scale scenes should be considered by, for example, adaptively choosing grid resolution. Another interesting topic would be on-demand, lazy construction of trees during the tracing phase through the use of the GBVH framework.

Acknowledgements Models used are courtesy of their creators. The dragon model is from the Stanford 3D Scanning Repository, the conference model is from the McGuire Computer Graphics Archive, and the fairy and toasters scenes are from the Utah 3D Animation Repository.

References

1. Goldsmith, J., Salmon, J.: Automatic creation of object hierarchies for ray tracing. IEEE Comput. Graph. Appl. **7**(5), 14–20 (1987)
2. Lext, J., Akenine-Möller, T.: Towards rapid reconstruction for animated ray tracing. In: EUROGRAPHICS 2001 Short Presentations (2001)
3. Parker, S.G., Bigler, J., Dietrich, A., Friedrich, H., Hoberock, J., Luebke, D., McAllister, D., McGuire, M., Morley, K., Robison, A., Stich, M.: OptiX: a general purpose ray tracing engine. ACM Trans. Graph. **29**(4), 66:1-66:13 (2010)
4. Wald, I., Woop, S., Benthin, C., Johnson, G.S., Ernst, M.: Embree: a kernel framework for efficient CPU ray tracing. ACM Trans. Graph. **33**(4), 1431–1438 (2014)
5. Wald, I., Boulos, S., Shirley, P.: Ray tracing deformable scenes using dynamic bounding volume hierarchies. ACM Trans. Graph. **26**(1), 61–618 (2007)
6. Wald, I.: On fast construction of SAH-based bounding volume hierarchies. In: Proceedings of the 2007 IEEE Symposium on Interactive Ray Tracing, pp. 33–40 (2007)
7. Lauterbach, C., Garland, M., Sengupta, S., Luebke, D.P., Manocha, D.: Fast BVH construction on GPUs. Comput. Graph. Forum **28**(2), 375–384 (2009)
8. Larsson, T., Akenine-Möller, T.: A dynamic bounding volume hierarchy for generalized collision detection. Comput. Graph. **30**(3), 450–459 (2006)
9. Yoon, S.E., Curtis, S., Manocha, D.: Ray tracing dynamic scenes using selective restructuring. In: Proceedings of the 18th Eurographics Conference on Rendering Techniques, pp. 73–84 (2007)
10. Kopta, D., Ize, T., Spjut, J., Brunvand, E., Davis, A., Kensler, A.: Fast, effective BVH updates for animated scenes. In: Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games, pp. 197–204 (2012)
11. Lauterbach, C., Yoon, S.E., Tuft, D., Manocha, D.: RT-DEFORM: interactive ray tracing of dynamic scenes using BVHs. In: 2006 IEEE Symposium on Interactive Ray Tracing, pp. 39–46 (2006)
12. Wald, I., Mark, W.R., Günther, J., Boulos, S., Ize, T., Hunt, W., Parker, S.G., Shirley, P.: State of the art in ray tracing animated scenes. Comput. Graph. Forum **28**(6), 1691–1722 (2009)
13. Fujimoto, A., Tanaka, T., Iwata, K.: ARTS: accelerated ray-tracing system. IEEE Comput. Graph. Appl. **6**(4), 16–26 (1986)
14. Amanatides, J., Woo, A.: A fast voxel traversal algorithm for ray tracing. In: Proceedings of Eurographics '87, pp. 3–10 (1987)
15. Jevans, D., Wyvill, B.: Adaptive voxel subdivision for ray tracing. In: Proceedings of Graphics Interface '89, pp. 164–172 (1989)
16. Reinhard, E., Smits, B., Hansen, C.: Dynamic acceleration structures for interactive ray tracing. In: Rendering Techniques, pp. 299–306 (2000)
17. Pantaleoni, J., Luebke, D.: HLBVH: hierarchical LBVH construction for real-time ray tracing of dynamic geometry. In: Proceedings of the Conference on High Performance Graphics, HPG '10, pp. 87–95 (2010)
18. Garanzha, K., Premože, S., Bely, A., Galaktionov, V.: Grid-based SAH BVH construction on a GPU. Vis. Comput. **27**, 697–706 (2011)
19. Ganestam, P., Barringer, R., Doggett, M., Akenine-Möller, T.: Bonsai: rapid bounding volume hierarchy generation using mini trees. J. Comput. Graph. Technol. **4**(3), 23–42 (2015)
20. Karras, T., Aila, T.: Fast parallel construction of high-quality bounding volume hierarchies. In: Proceedings of the 5th High-Performance Graphics Conference, HPG '13, pp. 89–99 (2013)
21. Domingues, L.R., Pedrini, H.: Bounding volume hierarchy optimization through agglomerative treelet restructuring. In: Proceedings of the 7th Conference on High-Performance Graphics, HPG '15, pp. 13–20 (2015)
22. Walter, B., Bala, K., Kulkarni, M., Pingali, K.: Fast agglomerative clustering for rendering. In: 2008 IEEE Symposium on Interactive Ray Tracing, pp. 81–86 (2008)
23. Meister, D., Bittner, J.: Parallel locally-ordered clustering for bounding volume hierarchy construction. IEEE Trans. Vis. Comput. Graph **24**(3), 1345–1353 (2018)
24. Bittner, J., Hapala, M., Havran, V.: Incremental BVH construction for ray tracing. Comput. Graph. **47**(C), 135–144 (2015)
25. Wald, I., Havran, V.: On building fast kd-trees for ray tracing, and on doing that in O(N log N). In: 2006 IEEE Symposium on Interactive Ray Tracing, pp. 61–69 (2006)
26. Yang, X., Liu, Q., Zhang, P., Xin, L., Zhou, D., Wang, Y., Zhang, Q., Wei, X.: DKD: a fast k-d tree update design for dynamic scenes. Comp. Anim. Virtual Worlds **27**, 340–350 (2016)
27. Lext, J., Assarsson, U., Möller, T.: A benchmark for animated ray tracing. IEEE Comput. Graph. Appl. **21**(2), 22–31 (2001)

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Satoshi Nishimura received the B.E. degree from Tohoku University in 1987, and the M.Sc. and D.Sc. degrees in information science from the University of Tokyo in 1989 and 1995, respectively. He is currently a senior associate professor at the University of Aizu. His research interests include computer graphics and computer music. He is a member of IEEE, ACM, and IPSJ.